

2 DOF Tilt Plate

Team Members:

Jasper Tan

Summary:

For my final project I built a 2 DOF Tilt plate project in which a steel ball rolls on the top plate to a specific set point. The steel ball rolls in reaction to the tilt plate tilting at a certain degree in the X and Y directions. The steel ball sits atop a resistive touch plate which produces XY coordinates of the ball. Based on the ball location, the servo motors controlling the plate tilt moves the ball into the specific set point. In addition to moving the ball to a set point, the tilt plate is programmed to move the ball in certain patterns in order to fulfill the Tier 2 Option B requirement. The tilt plate moves the ball in a square trajectory or a circle trajectory. Set point, ball trajectory patterns, PID gains, and servo limits are all settable in a GUI made for this project as well.

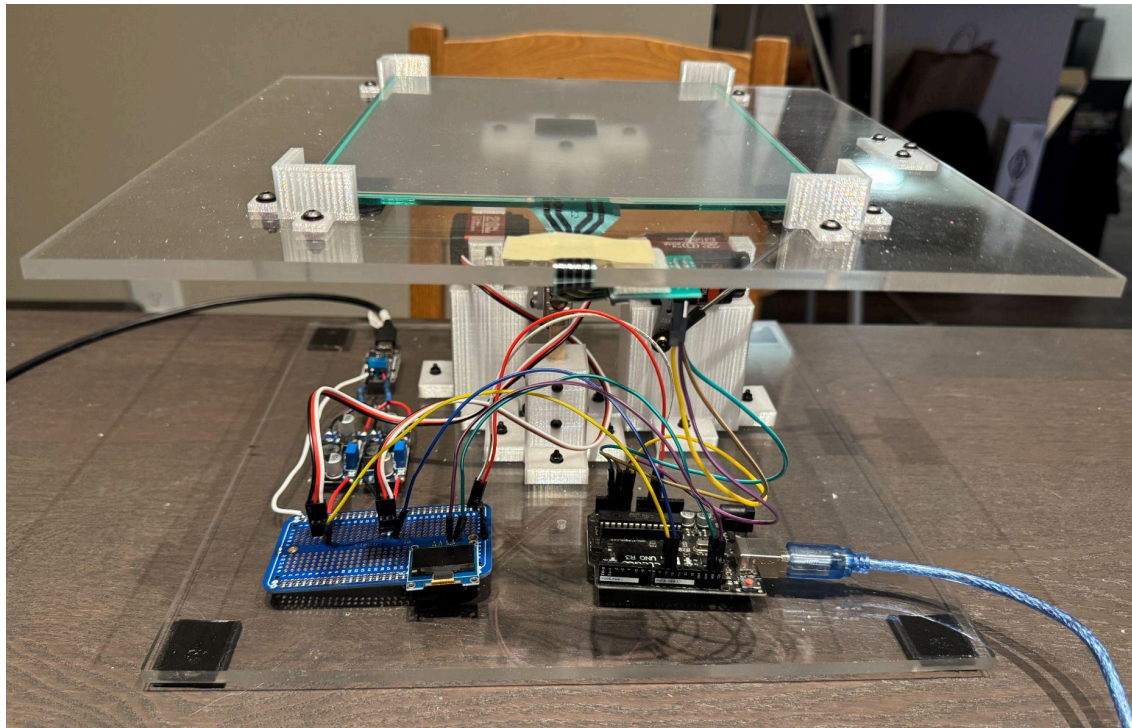


Figure 1: Tilt Plate Project Photo

System Overview:

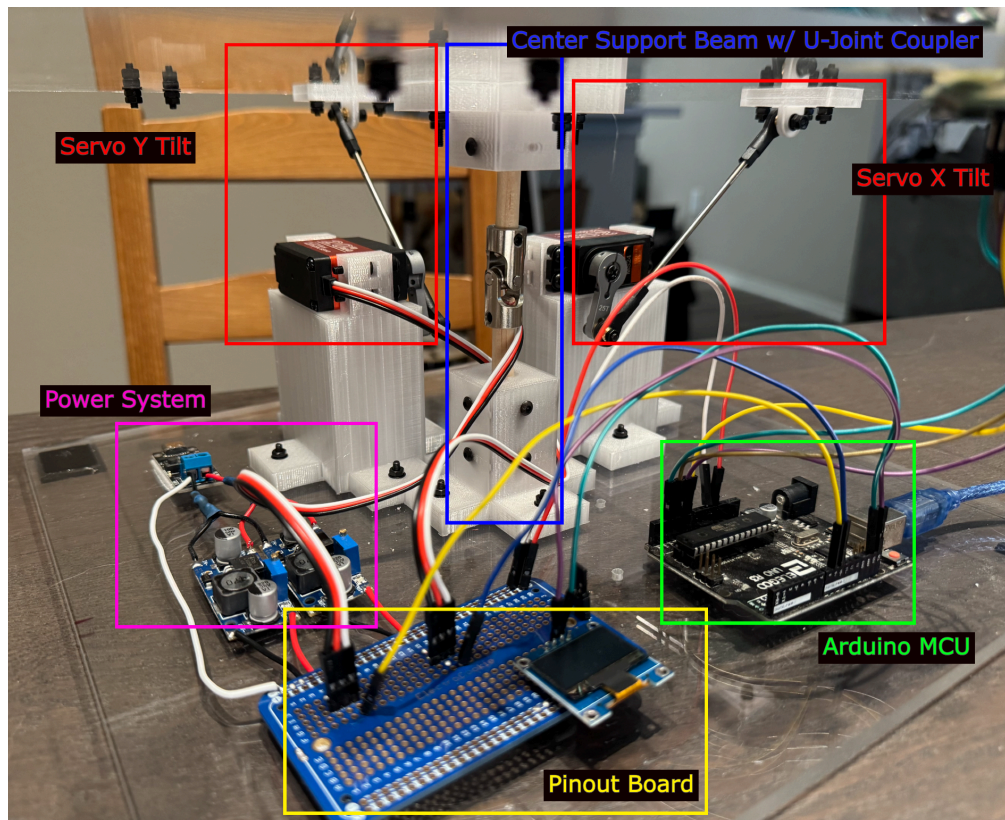


Figure 2: System Overview - Bottom Plate Labeled

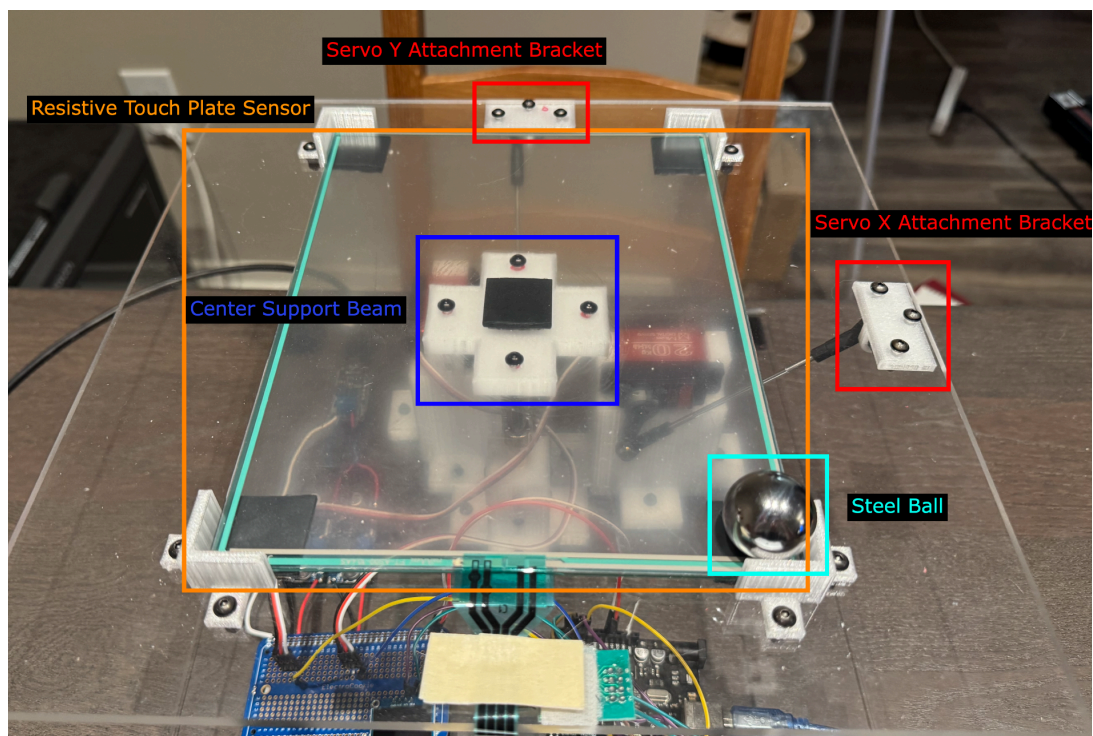


Figure 3: System Overview - Top Plate Labeled

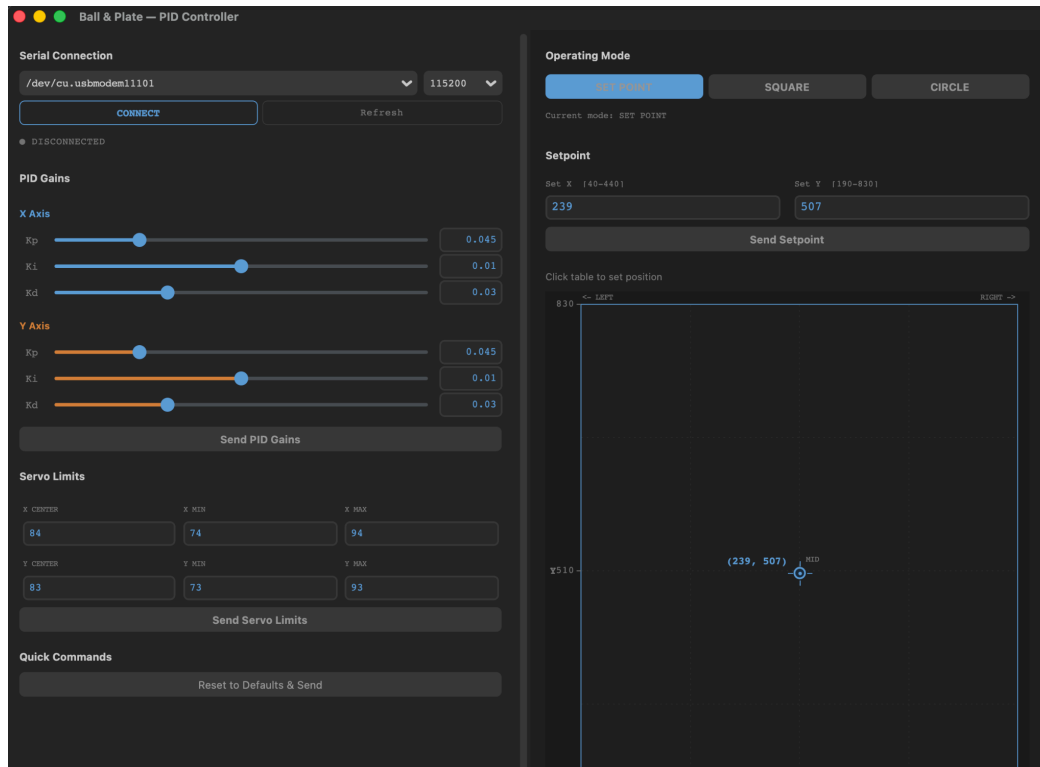


Figure 4: System GUI - Contains sections for PID Gain configuration, Servo Limit setting, Operating Mode of Set Point, Square, or Circle, and explicit set point setting with clickable tilt plate visualization

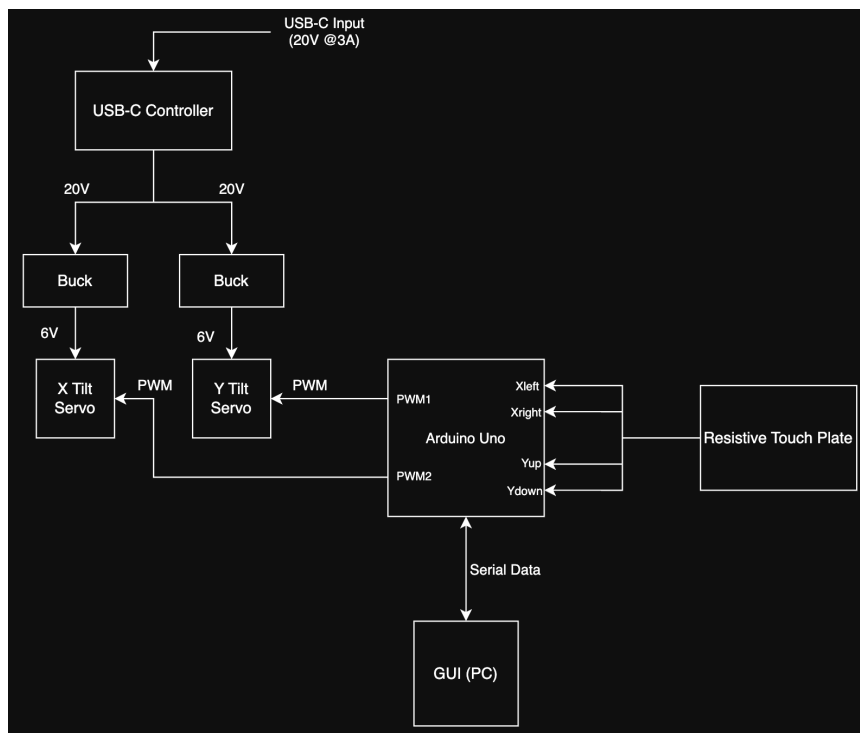
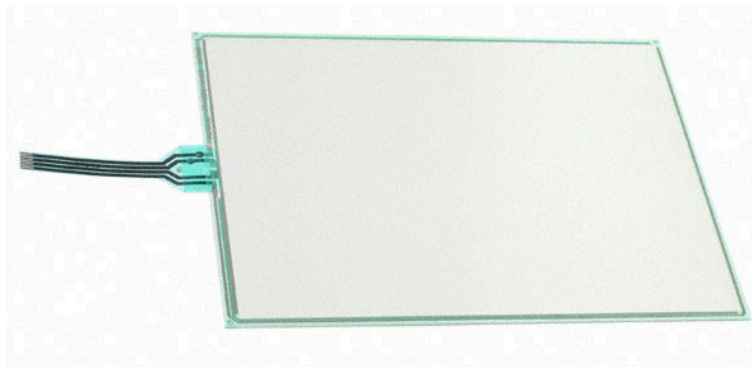


Figure 5: High Level Component Block Diagram

Design Description:

Sensor: The sensor used is a resistive touch plate sensor. This touch plate consists of two conductive layers separated by a small gap. Upon pressing the touch plate (with the steel ball) the layers make contact at that point. By applying voltage across one layer and measuring the voltage at the contact points on the other, the system can determine the touch position based on the resulting voltage divider. This process can be repeated in both X and Y directions to get the full 2D coordinates of the touch point. The resistive touch plate has extremely fast response as it is simply a voltage divider. Reading the resistive touch plate is limited only to the Microcontroller GPIO applying voltage to one layer, and using the ADC to read the voltage divider on the other layer. The measured signal itself is affected by noise due to contact variability. To fix this, a low pass filter was implemented to reduce reading spikes and jitter.

Part#: FTAS00-104AS4 ([Datasheet Link](#))



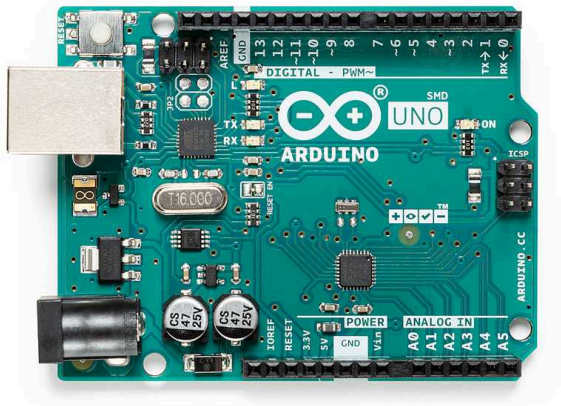
Actuator: The actuators used are two servo motors. These digital servo motors are high torque (22.8kg*cm (316.63oz*in)) with a wide control angle of 270 degrees. They are primarily used in this application to attach to connecting rods attached to the top plate. These servo motors then control the tilt of the top plate. Prior to attaching the connecting rods, the servo motors must be calibrated by programming to 90 degrees, then attaching the servo horn and connecting rods at the desired angle. Reason in doing so is without first programming to an initial angle, position of servo motor is indeterminate and programming a set angle to the servo may produce an unexpected angle on the tilt table X or Y direction. These servo motors are limited in my Arduino program to a certain angle such that the tilt table can only tilt +/- 4 degrees.

Part#: DS3218 ([Datasheet Link](#))



Microcontroller: The microcontroller used to implement the program logic as well as control loop is the Arduino Uno board which contains the Atmega328p, a 8 bit MCU. The MCU was primarily used to read resistive touch plate data via ADC, set angles for the X and Y servo motors, and implement the PID control loop.

Part#: Arduino Uno ([Datasheet Link](#))



Overall Schematic:

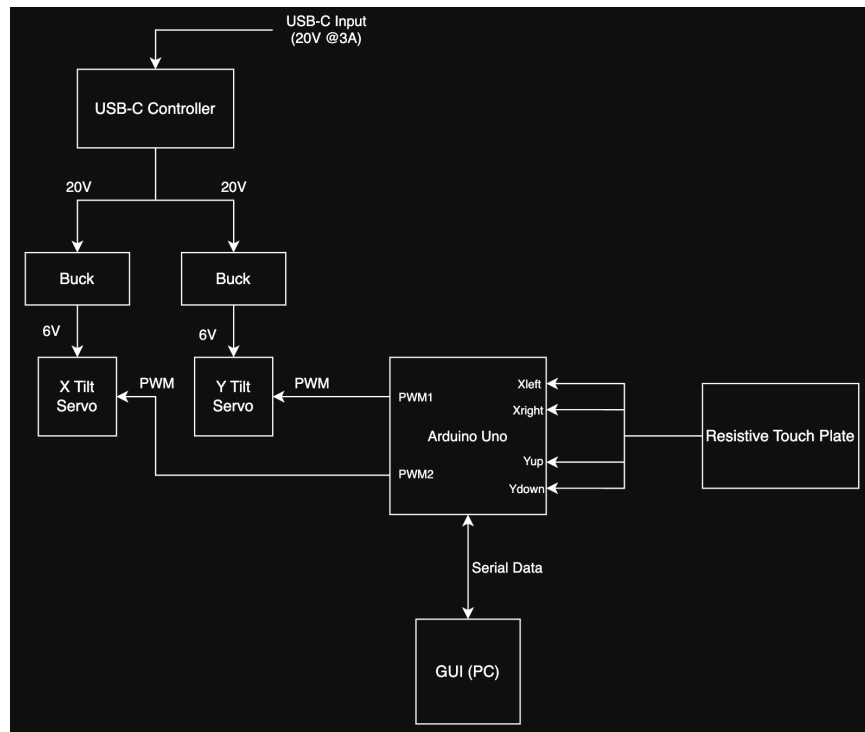


Figure 6: Overall Schematic

Software Design:

The control program implements a closed loop PID controller for the two axis tilt plate system. It controls two servos with resistive touch sensor inputs to track the ball position in X and Y directions. The control loop has a 10ms period, in which the Arduino reads the sensor coordinates, validates the readings using range and jump checks, and replaces invalid measurements with the last known good values to improve robustness. Range and jump checks are crucial in which occasionally during ball movement, the tilt plate does not have good contact, therefore upon reading the tilt plate indicates the ball has moved at an unrealistic speed to an unrealistic location. Therefore if movement occurs within a certain range, the movement is valid, else invalid measurements are replaced with the last known good values. In addition, all ADC readings are averaged to reduce noise as well. Based on the current operating mode, the controller either sets a fixed setpoint, a square trajectory, or a circular trajectory for the ball to follow. The PID algorithm then calculates proportional, integral, and derivative control terms for each of the X and Y axis tilts. This includes protection to prevent windup for the integral term as well as a low pass filter for the derivative term to reduce sensitivity to noise. The resulting control outputs are converted into servo angle commands which are constrained to motion limits restricting the tilt plate to tilt +/- 4 degrees at maximum. These servo angles are then sent to the X and Y servos to tilt the plate, driving the ball towards the desired position. Serial commands sent from the accompanied GUI are also read in Arduino and commands are parsed accordingly. From Serial commands, the PID terms of each axis, setpoints, and operating mode can all be set in real time.

Software link:

https://github.com/jaspertan10/2DOF_TiltTable/blob/main/Code/tiltplate_main/tiltplate_main.ino

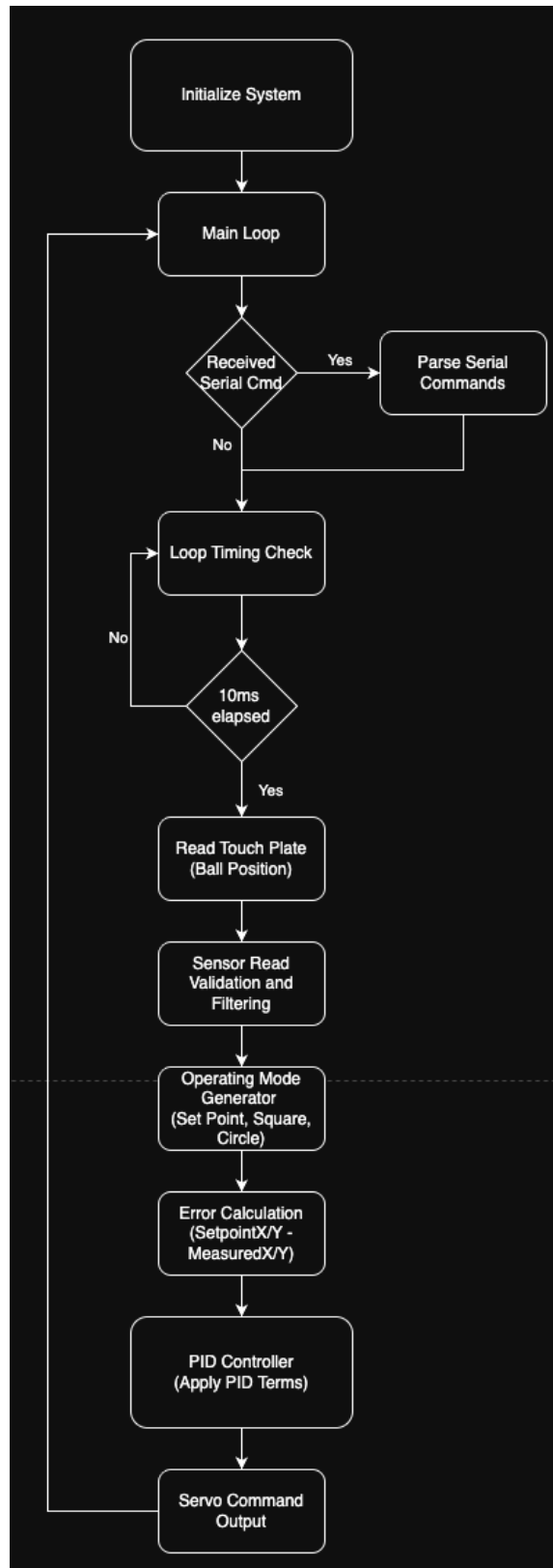


Figure 7: Software Block Diagram

Control System Design and Performance

Plant Model

The system is a two axis ball on plate in which the controlled outputs are the ball positions in the X and Y directions. Inputs to this system is the plate tilt angle (referred to as Θ_x and Θ_y) which is generated by two servo motors attached to the tilt plate, controlling Θ_x and Θ_y tilt, of which the tilt is limited to ± 4 degrees in both axis.

The plant dynamics are determined by the steel ball on the tilt plate with various forces acting upon it. Specifically gravity which provides the primary moving force, along with translational and rotational inertia of the ball.

Forces on the ball equation:

$$F_g = F_t + F_r$$

Where F_t = translational inertia, F_r = rotational inertia, and F_g = force of gravity. Where $F_t = m\ddot{x}$, $F_r = \frac{2}{5} m\ddot{\theta}$, and $F_g = m g \sin(\Theta)$, plugging these force equations makes:

$$m g \sin(\Theta) = m \ddot{x} + \frac{2}{5} m \ddot{\theta}$$

Solve for $\ddot{x}$ and we get:

EOM for X and Y axis:

$$\ddot{x} = \frac{5}{7} g \sin(\Theta_x)$$

$$\ddot{y} = \frac{5}{7} g \sin(\Theta_y)$$

For the purposes of a simplified model, we will assume that rolling resistances such as friction and air resistance are negligible and therefore omitted from our equation of motion. In addition for simplicity, since tilt angles are small i.e. ± 4 degrees max, we can generalize $\sin(\Theta) \approx \Theta$. For example, 4 degrees = .07 rad, $\sin(4 \text{ degrees}) = .07$. Therefore our new equation of motion can become:

$$\ddot{x} = \frac{5}{7} g \Theta_{x_rad}$$

$$\ddot{y} = \frac{5}{7} g \Theta_{y_rad}$$

If we wish to input in degrees our equation will become:

$$\ddot{x} = \frac{5}{7} g \Theta_x \frac{\pi}{180}$$

$$\ddot{y} = \frac{5}{7} g \Theta_y \frac{\pi}{180}$$

Simplifying these equations out becomes:

$$\ddot{x} = .1223 \cdot \Theta_x$$

$$\ddot{y} = .1223 \cdot \Theta_y$$

Transfer functions:

$$x(s)/\Theta x(s) = .1223 / s^2$$

$$y(s)/\Theta y(s) = .1223 / s^2$$

Sensor Model

As previously mentioned the system uses a resistive touch plate in order to measure the ball position in both X and Y axis. The sensor works by applying voltage to one conductive layer and upon pressing the touch plate, creates a voltage divider in the other conductive layer. Measuring this conductive layer with an ADC, produces a specific voltage level depending on where the ball has pressed. There is however some noise to the sensor, the voltage divider itself can fluctuate values, the ball is never 100% still, and the ADC also has some noise inherently. Therefore in software in order to mitigate noise ADC values are averaged, in addition to other protections outlined in the Software Design area of this report.

Actuator Model

As previously mentioned the system uses two servo motors as actuators, both of which control the plate tilt in the X and Y directions. The servos themselves are calibrated/initialized to a neutral position (flat plate) in which the servo programmed values are $X_{neutral} = 84$ and $Y_{neutral} = 83$. Both X and Y commanded servo angles are limited to ± 10 which translates to a plate tilt of ± 4 degrees, it is approximately a linear translation between the servo angle and the corresponding plate tilt angle, therefore for the sake of simplicity, will be assumed linear.

Simulink Modeling

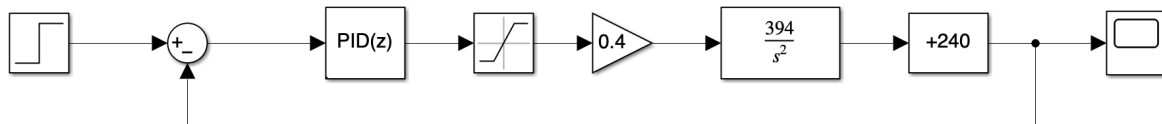


Figure 8. Simulink Model of Tilt Table X axis

A single X axis simulink model was developed as shown in Figure 8 above in order to estimate the behavior of the implemented control system. This model uses the exact PID gains (0.045, 0.01, 0.03, respectively) in the actual tilt table system as well as sample time (10ms), tilt table angle (± 4 degrees), and servo command limits (74 to 94). Similar to the actual tilt table, position and error is calculated by ADC counts produced by the resistive touch sensor.

For the transfer function itself, in the original formula provided assumed an output in meters, however the implemented control system operates on ADC counts, especially since the resistive touch sensor provides position data in this form. Therefore the plant model was altered to reflect the conversion from meter output to ADC counts.

In order to convert from meter output to ADC counts output, the range of ADC values were mapped to the X width of the tilt plate. The X width of the tilt plate was 6.236 inches (0.1584

meters), with readable ADC range between 40-550. On scaling to 1 meter, there would be approximately 3219 ADC counts per meter.

So the new $K = 0.1223 * 3219 = 394$

Therefore the new transfer function which outputs ADC count is:

$$X_{adc}(s)/\Theta(s) = 394 / s^2$$

Output $X(s)$ is now in ADC counts while the input remains in degrees ($\Theta(s)$). Thus the simulink is a direct representation of the actual tilt table system.

Simulink Output

Step input: Initial 240, Final 300

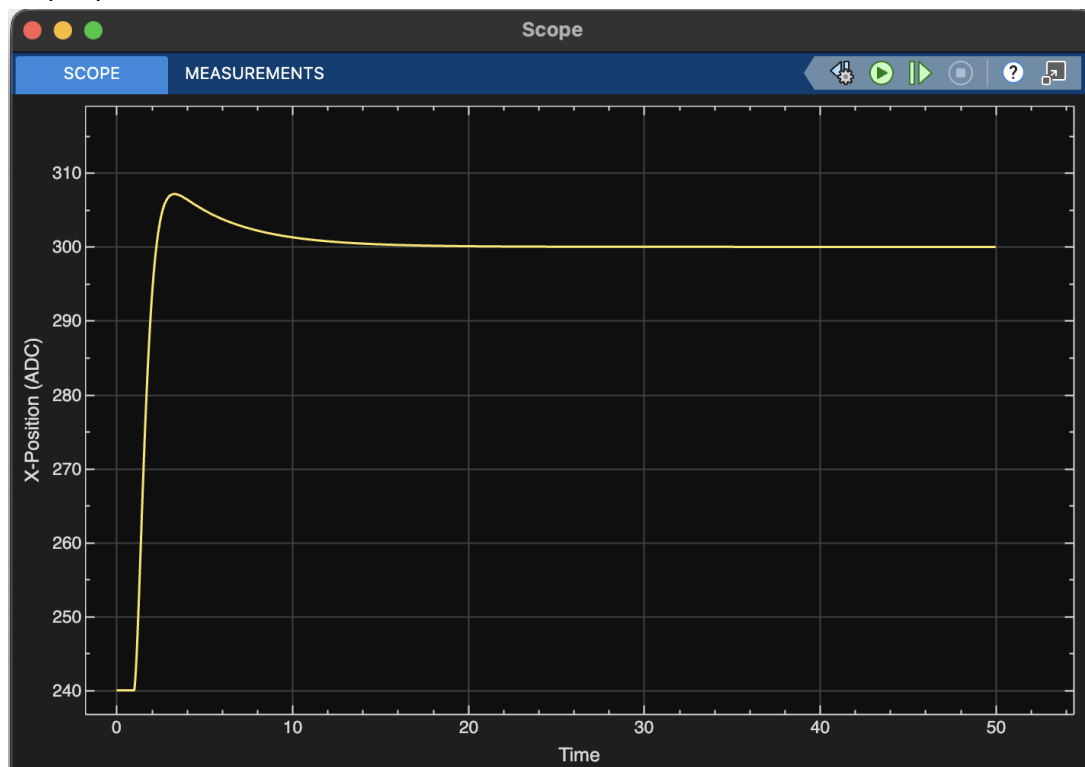


Figure 9: Simulated scope of X Position (ADC) with step from 240 to 300

As shown in Figure 9 above we have a high K_p therefore we overshoot past the setpoint at ~2 seconds and slowly reach steady state within ~15 seconds.

Actual Tilt Table Performance Output

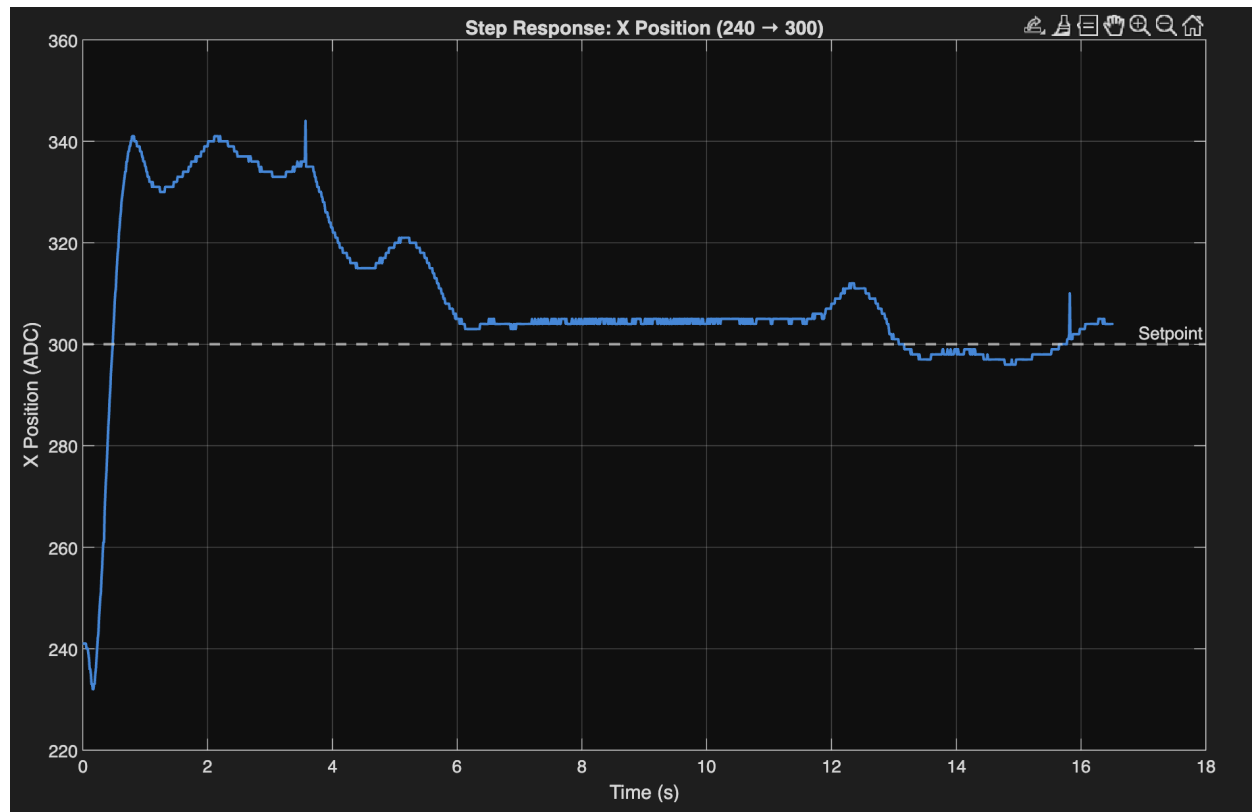


Figure 10: Tilt Table Plot of X Position (ADC) with setpoint from 240 to 300

As shown in Figure 10, the tilt table response is generally similar to the Simulink model in Figure 9, though with increased noise and considerably higher overshoot. In the simulated response, the system overshoots to approximately 310 ADC counts, whereas the experimental data shows overshoot reaching approximately 340 ADC counts. Also, the tilt table response does not reach steady state and stay there, instead it continues to fluctuate around the setpoint.

The discrepancies between simulink and tilt table response graphs can primarily be attributed to issues with the resistive touch plate and the steel ball. The resistive touch plate has measurement noise and limited accuracy, in particular due to the use of a 1 inch diameter steel ball. Because the ball has a relatively large contact area, the touch point is not fixed and can vary, especially as the ball rolls. Furthermore, during motion, contact between the ball and the touch plate is not consistent, leading to incorrect or unreliable readings. This results of course in fluctuations in the read ADC data of the tilt table, even when the ball is near the set point. As a result, the system appears noisier and not at steady state, especially compared to the idealized simulink simulation.

The simulation results and transfer function presented previously are specific to the X axis dynamics, however, the same modeling approach can be directly applied to the Y axis as well, as the system exhibits similar behavior in both directions.